



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Práctica de laboratorio 1: Creación de procesos e hilos en C y Java

Heriberto Espino Montelongo

ID: 175199

Andrea Hernandez Huerta

ID: 175523

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 1 de septiembre de 2026

Índice general

Índice de figuras	III
Lista de abreviaturas y símbolos	IV
1. Creación de un proceso en C	1
2. Creación de procesos en Java	3
3. Creación de un hilo en C	4
4. Creación de un hilo en Java	6
5. Múltiples hilos en Java	9
6. Comparación entre procesos e hilos en C y Java	10
Referencias	11

Índice de figuras

1.1. Código fuente y ejecución del proceso en C.	2
2.1. Código fuente y ejecución del proceso en Java con ProcessBuilder.	3
3.1. Código fuente y ejecución del hilo en C.	4
4.1. Código fuente y ejecución del hilo en Java.	7
4.2. Segunda ejecución del hilo en Java.	8
5.1. Código fuente y ejecución de los múltiples hilos en Java. . .	9

Lista de abreviaturas y símbolos

CPU *Central Processing Unit*: unidad central de procesamiento.

JVM *Java Virtual Machine*: máquina virtual que ejecuta el bytecode de Java y permite que los programas se ejecuten sobre distintos sistemas operativos.

OS *Operating System*: sistema operativo.

PID *Process Identifier*: identificador del proceso.

PPID *Parent Process Identifier*: identificador del proceso padre.

POSIX *Portable Operating System Interface*: conjunto de estándares que define interfaces y servicios comunes para sistemas operativos tipo Unix, como las funciones `fork()` y `wait()`.

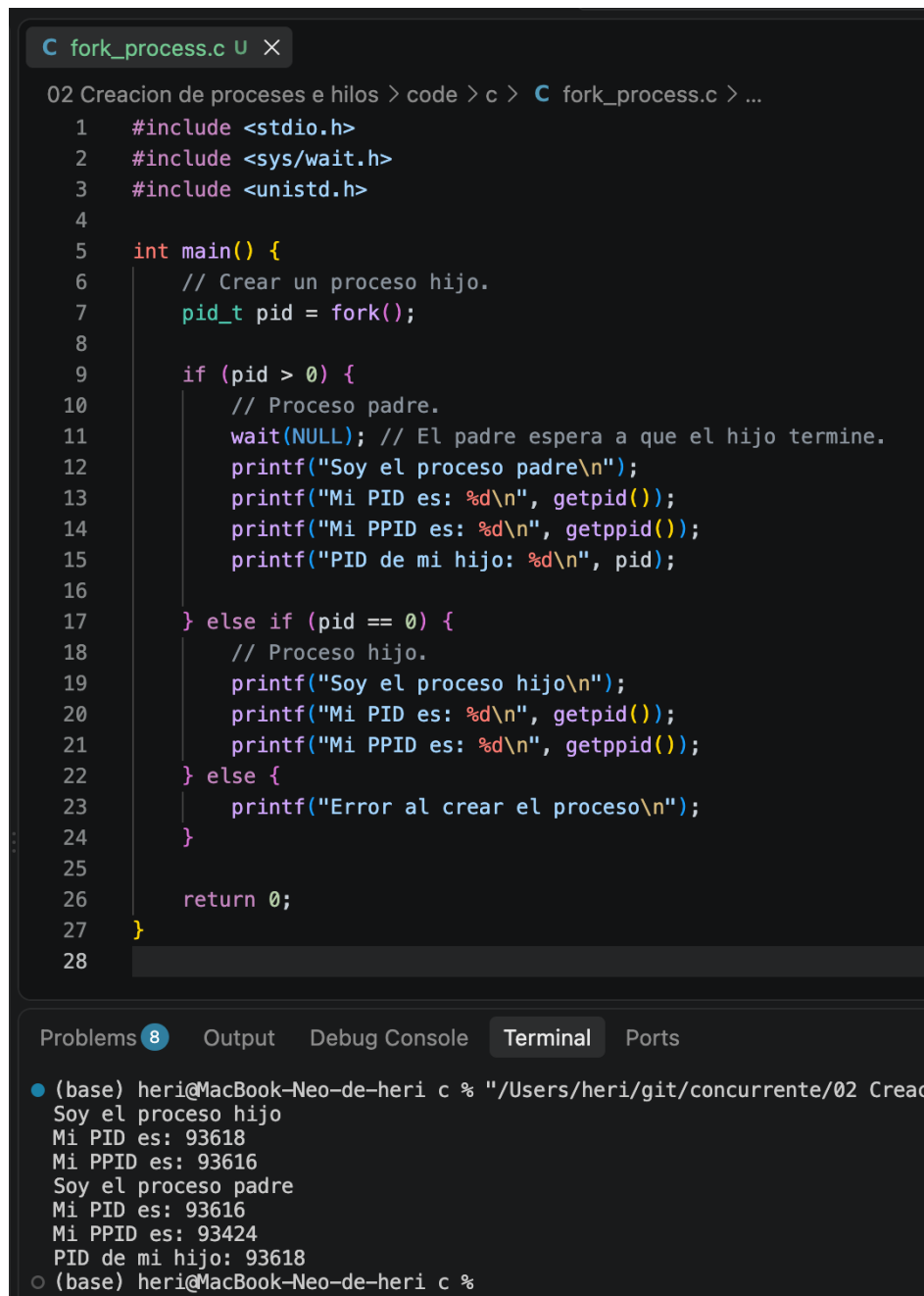
1. Creación de un proceso en C

Desarrolla un programa que utilice `fork()` para crear un proceso hijo. El proceso padre deberá esperar al hijo utilizando `wait()`.

En la [Figura 1.1](#), el proceso hijo aparece primero porque el proceso padre ejecuta `wait(NULL)` inmediatamente después de `fork()`. Mientras el padre está bloqueado, el hijo imprime su mensaje, su PID y el PID de su padre, y termina.

Cuando el hijo termina, `wait(NULL)` permite que el padre continúe y después imprima sus propios datos. Por eso, el orden es primero hijo y después padre. Además, el PPID que muestra el hijo coincide con el PID del padre, mientras que el PID que el padre muestra como *PID de mi hijo* coincide con el PID del hijo.

Si se elimina `wait()`, el proceso padre ya no espera explícitamente al hijo y ambos pueden imprimir sus mensajes en un orden diferente. El padre podría terminar antes que el hijo. Por lo tanto, `wait()` sincroniza al padre con el hijo.



```
C fork_process.c U X
02 Creacion de procesos e hilos > code > c > C fork_process.c > ...
1  #include <stdio.h>
2  #include <sys/wait.h>
3  #include <unistd.h>
4
5  int main() {
6      // Crear un proceso hijo.
7      pid_t pid = fork();
8
9      if (pid > 0) {
10         // Proceso padre.
11         wait(NULL); // El padre espera a que el hijo termine.
12         printf("Soy el proceso padre\n");
13         printf("Mi PID es: %d\n", getpid());
14         printf("Mi PPID es: %d\n", getppid());
15         printf("PID de mi hijo: %d\n", pid);
16
17     } else if (pid == 0) {
18         // Proceso hijo.
19         printf("Soy el proceso hijo\n");
20         printf("Mi PID es: %d\n", getpid());
21         printf("Mi PPID es: %d\n", getppid());
22     } else {
23         printf("Error al crear el proceso\n");
24     }
25
26     return 0;
27 }
28
```

Problems 8 Output Debug Console Terminal Ports

```
● (base) heri@MacBook-Neo-de-heri c % "/Users/heri/git/concurrente/02 Creacion de procesos e hilos"
Soy el proceso hijo
Mi PID es: 93618
Mi PPID es: 93616
Soy el proceso padre
Mi PID es: 93616
Mi PPID es: 93424
PID de mi hijo: 93618
○ (base) heri@MacBook-Neo-de-heri c %
```

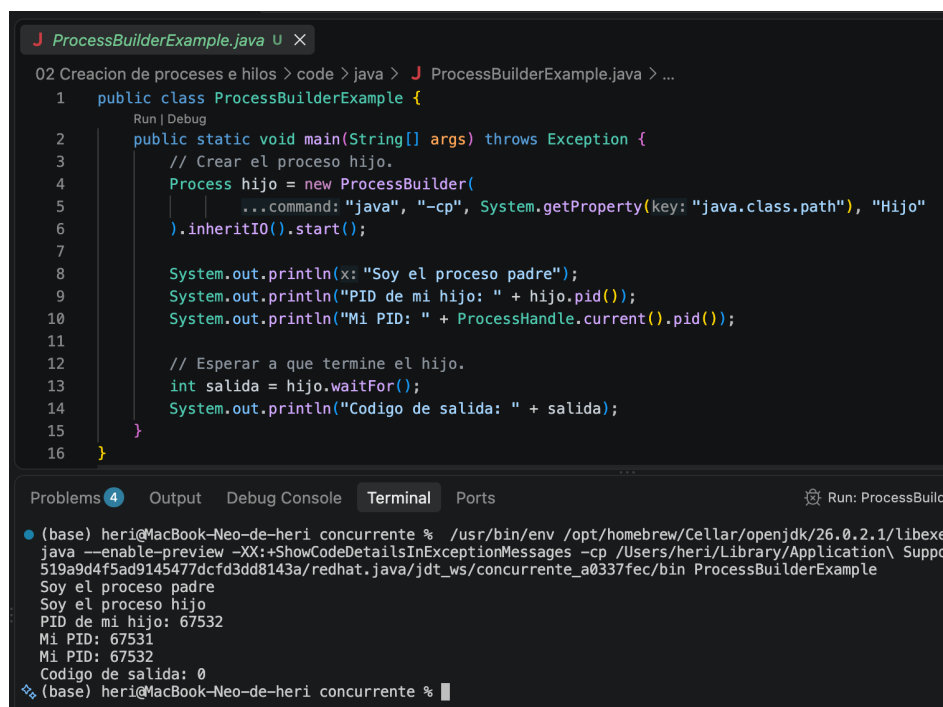
Figura 1.1: Código fuente y ejecución del proceso en C.

2. Creación de procesos en Java

Desarrolla un programa que use `ProcessBuilder` para iniciar un proceso hijo.

La Figura 2.1 muestra el código y la ejecución del proceso hijo iniciado mediante `ProcessBuilder`.

`fork()` es una llamada POSIX que crea un proceso hijo como copia del proceso actual. Después de la llamada, padre e hijo continúan desde el mismo punto del programa, pero reciben valores de retorno distintos. En cambio, `ProcessBuilder` es una API de Java que configura y ejecuta un comando externo; no duplica directamente el estado de la JVM. Por esta razón, `ProcessBuilder` es una abstracción de más alto nivel y depende menos de detalles específicos de POSIX.



```
J ProcessBuilderExample.java U X
02 Creacion de procesos e hilos > code > java > J ProcessBuilderExample.java > ...
1 public class ProcessBuilderExample {
2     // Crear el proceso hijo.
3     Process hijo = new ProcessBuilder(
4         ...command: "java", "-cp", System.getProperty(key: "java.class.path"), "Hijo"
5     ).inheritIO().start();
6
7     System.out.println(x: "Soy el proceso padre");
8     System.out.println("PID de mi hijo: " + hijo.pid());
9     System.out.println("Mi PID: " + ProcessHandle.current().pid());
10
11     // Esperar a que termine el hijo.
12     int salida = hijo.waitFor();
13     System.out.println("Codigo de salida: " + salida);
14 }
15
16

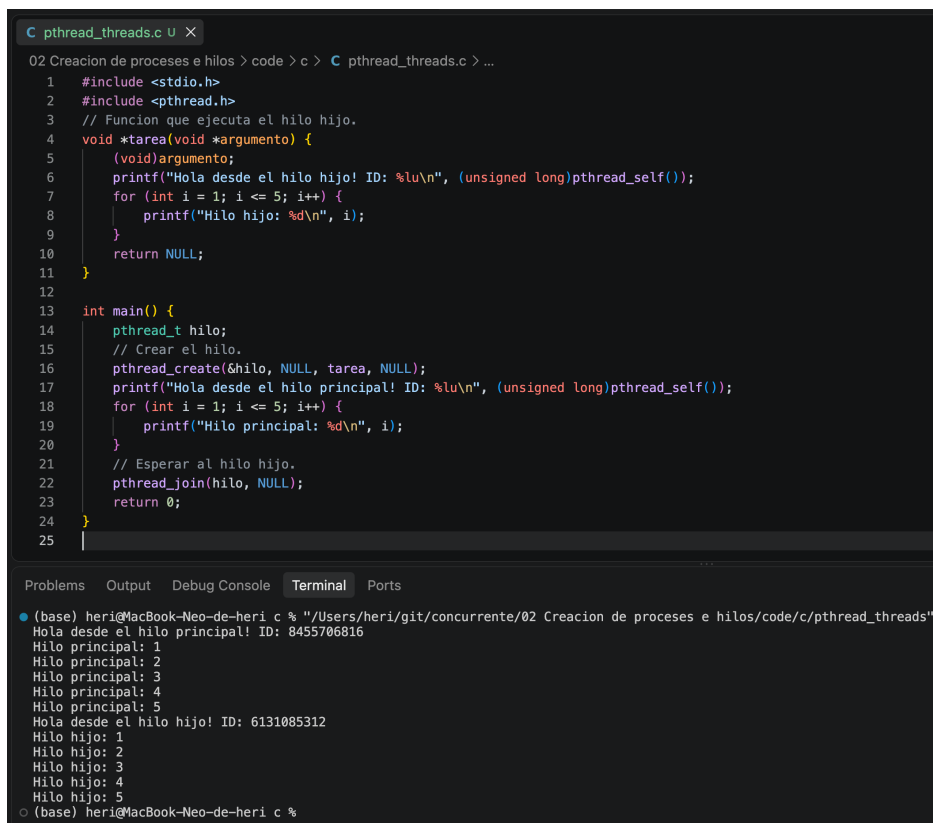
Problems 4 Output Debug Console Terminal Ports Run: ProcessBuild
(base) heri@MacBook-Neo-de-heri concurrente % /usr/bin/env /opt/homebrew/Cellar/openjdk/26.0.2.1/libexec
java --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/heri/Library/Application\ Suppo
519a9d4f5ad9145477dcfd3dd8143a/redhat.java/jdt_ws/concurrente_a0337fec/bin ProcessBuilderExample
Soy el proceso padre
Soy el proceso hijo
PID de mi hijo: 67532
Mi PID: 67531
Mi PID: 67532
Codigo de salida: 0
(base) heri@MacBook-Neo-de-heri concurrente %
```

Figura 2.1: Código fuente y ejecución del proceso en Java con `ProcessBuilder`.

3. Creación de un hilo en C

Utiliza `pthread_create()` para crear un hilo hijo. El hilo principal deberá esperar al hijo mediante `pthread_join()`.

La Figura 3.1 muestra la ejecución del hilo principal y del hilo hijo creado con `pthread_create()`.



```
C pthread_threads.c U X
02 Creacion de procesos e hilos > code > c > C pthread_threads.c > ...
1  #include <stdio.h>
2  #include <pthread.h>
3  // Funcion que ejecuta el hilo hijo.
4  void *tarea(void *argumento) {
5      (void)argumento;
6      printf("Hola desde el hilo hijo! ID: %lu\n", (unsigned long)pthread_self());
7      for (int i = 1; i <= 5; i++) {
8          printf("Hilo hijo: %d\n", i);
9      }
10     return NULL;
11 }
12
13 int main() {
14     pthread_t hilo;
15     // Crear el hilo.
16     pthread_create(&hilo, NULL, tarea, NULL);
17     printf("Hola desde el hilo principal! ID: %lu\n", (unsigned long)pthread_self());
18     for (int i = 1; i <= 5; i++) {
19         printf("Hilo principal: %d\n", i);
20     }
21     // Esperar al hilo hijo.
22     pthread_join(hilo, NULL);
23     return 0;
24 }
25
```

Problems Output Debug Console Terminal Ports

```
• (base) heri@MacBook-Neo-de-heri c % "/Users/heri/git/concurrente/02 Creacion de procesos e hilos/code/c/pthread_threads"
Hola desde el hilo principal! ID: 8455706816
Hilo principal: 1
Hilo principal: 2
Hilo principal: 3
Hilo principal: 4
Hilo principal: 5
Hola desde el hilo hijo! ID: 6131085312
Hilo hijo: 1
Hilo hijo: 2
Hilo hijo: 3
Hilo hijo: 4
Hilo hijo: 5
○ (base) heri@MacBook-Neo-de-heri c %
```

Figura 3.1: Código fuente y ejecución del hilo en C.

En las ejecuciones realizadas se observó el mismo orden de los mensajes. Esto puede ocurrir porque el programa es muy pequeño, los ciclos terminan rápidamente y el sistema operativo suele tomar decisiones parecidas cuando la computadora se encuentra bajo una carga similar. Sin embargo, ese orden no está garantizado: el planificador decide qué hilo recibe tiempo de CPU en cada instante, por lo que los mensajes podrían intercalarse de otra manera

Ejercicio 3. Creación de un hilo en C

bajo diferentes condiciones. La llamada `pthread_join()` garantiza que el hilo principal espere la terminación del hilo hijo, pero no fija el orden en que ambos hilos imprimen sus mensajes.

4. Creación de un hilo en Java

Crea un hilo mediante la clase Thread. Ejecuta el programa varias veces y registra el orden de los mensajes.

La salida mostrada en las [Figura 4.1](#) y [Figura 4.2](#) puede cambiar entre ejecuciones porque el hilo principal y el hilo secundario se ejecutan de manera concurrente. El planificador de la JVM y del sistema operativo puede alternar entre ambos en momentos diferentes en cada ejecución. La llamada `Thread.sleep()` pausa al hilo actual, pero no garantiza cuál hilo continuará primero al terminar la pausa. Por eso las líneas pueden aparecer en un orden distinto.

The image shows a code editor with a file named `ThreadExample.java`. The code defines a `ThreadExample` class with a `main` method. Inside `main`, a secondary thread is created and started, which prints "Hilo secundario" five times. The main thread prints "Hilo principal" five times. Both threads include a `pausa()` method that sleeps for 500 milliseconds. The `main` method uses `hilo.join()` to wait for the secondary thread to finish before continuing its loop.

```

1  public class ThreadExample {
2      private static void pausa() {
3          try {
4              Thread.sleep(500);
5          } catch (InterruptedException e) {
6              Thread.currentThread().interrupt();
7          }
8      }
9
10     Run | Debug
11     public static void main(String[] args) {
12         // Crear el hilo secundario.
13         Thread hilo = new Thread(() -> {
14             for (int i = 1; i <= 5; i++) {
15                 System.out.println(x: "Hilo secundario");
16                 pausa();
17             }
18         });
19         hilo.start();
20
21         for (int i = 1; i <= 5; i++) {
22             System.out.println(x: "Hilo principal");
23             pausa();
24         }
25
26         try {
27             // Esperar a que termine el hilo.
28             hilo.join();
29         } catch (InterruptedException e) {
30             Thread.currentThread().interrupt();
31         }
32     }
33 }
34

```

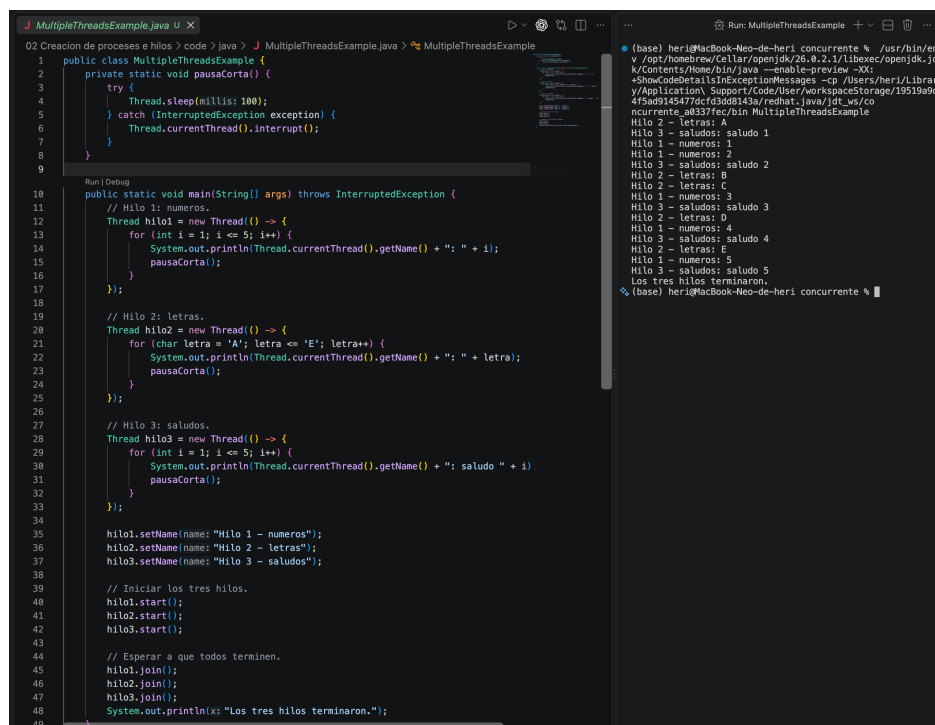
The terminal output shows the execution of the program. It prints "Hilo principal" and "Hilo secundario" five times each, with the secondary thread's output interleaved with the main thread's output. The terminal prompt is `(base) heri@MacBook-Neo-de-heri concurrente %`.

Figura 4.1: Código fuente y ejecución del hilo en Java.

5. Múltiples hilos en Java

Crea tres hilos que representen las siguientes tareas: Asigna un nombre a cada hilo mediante `setName()` y muestra su nombre utilizando `Thread.currentThread().getName()`.

El hilo principal deberá iniciar los tres hilos y esperar a que todos terminen utilizando `join()`.



```
02 Creacion de procesos e hilos > code > java > J MultipleThreadsExample.java > % MultipleThreadsExample
1 public class MultipleThreadsExample {
2     private static void pausaCorta() {
3         try {
4             Thread.sleep(100);
5         } catch (InterruptedException exception) {
6             Thread.currentThread().interrupt();
7         }
8     }
9
10    Run | Debug
11    public static void main(String[] args) throws InterruptedException {
12        // Hilo 1: numeros.
13        Thread hilo1 = new Thread() -> {
14            for (int i = 1; i <= 5; i++) {
15                System.out.println(Thread.currentThread().getName() + ": " + i);
16                pausaCorta();
17            }
18        };
19
20        // Hilo 2: letras.
21        Thread hilo2 = new Thread() -> {
22            for (char letra = 'A'; letra <= 'E'; letra++) {
23                System.out.println(Thread.currentThread().getName() + ": " + letra);
24                pausaCorta();
25            }
26        };
27
28        // Hilo 3: saludos.
29        Thread hilo3 = new Thread() -> {
30            for (int i = 1; i <= 5; i++) {
31                System.out.println(Thread.currentThread().getName() + ": saludo " + i);
32                pausaCorta();
33            }
34        };
35
36        hilo1.setName(name: "Hilo 1 - numeros");
37        hilo2.setName(name: "Hilo 2 - letras");
38        hilo3.setName(name: "Hilo 3 - saludos");
39
40        // Iniciar los tres hilos.
41        hilo1.start();
42        hilo2.start();
43        hilo3.start();
44
45        // Esperar a que todos terminen.
46        hilo1.join();
47        hilo2.join();
48        hilo3.join();
49        System.out.println(x: "Los tres hilos terminaron.");
50    }
51 }
```

```
(base) heri@MacBook-Neo-de-heri concurrente % ./usr/bin/en
v /opt/homebrew/cellar/openjdk/26.8.2-1/libexec/openjdk.jd
k/Contents/Home/bin/java --enable-preview -XX:
+ShowCodeDetailsInExceptionMessages -cp /Users/heri/.librar
y/Application\ Support/Code/User/workspaceStorage/15519db8
4f5ad914547dcfd3d8143a/redhat.java/jdt_ws/co
ncurrente_a0337fec/bin MultipleThreadsExample
Hilo 2 - letras: A
Hilo 3 - saludos: saludo 1
Hilo 1 - numeros: 1
Hilo 1 - numeros: 2
Hilo 3 - saludos: saludo 2
Hilo 2 - letras: B
Hilo 1 - numeros: 3
Hilo 3 - saludos: saludo 3
Hilo 2 - letras: C
Hilo 1 - numeros: 4
Hilo 3 - saludos: saludo 4
Hilo 2 - letras: D
Hilo 1 - numeros: 5
Hilo 3 - saludos: saludo 5
Hilo 2 - letras: E
Hilo 1 - numeros: terminaron.
Hilo 3 - saludos: terminaron.
Los tres hilos terminaron.
(base) heri@MacBook-Neo-de-heri concurrente %
```

Figura 5.1: Código fuente y ejecución de los múltiples hilos en Java.

La Figura 5.1 muestra la ejecución concurrente de las tres tareas y los nombres asignados a sus hilos.

El orden de las llamadas a `start()` no determina el orden en que los hilos reciben tiempo de CPU ni el orden en que terminan. Cada hilo puede avanzar a una velocidad diferente según el planificador y la carga del sistema. Las llamadas a `join()` solo garantizan que el hilo principal continúe cuando todos los hilos hayan terminado.

6. Comparación entre procesos e hilos en C y Java

Los procesos tienen un espacio de memoria independiente. Esto proporciona aislamiento entre programas, pero su creación y comunicación suele requerir más recursos. En C, `fork()` crea un proceso hijo a partir del proceso actual y `wait()` permite al padre esperar su terminación. En Java, `ProcessBuilder` inicia un programa externo y `waitFor()` espera su código de salida.

Los hilos comparten la memoria de un mismo proceso. Por ello son más ligeros que los procesos y pueden comunicarse mediante variables compartidas, aunque requieren sincronización para evitar condiciones de carrera. En C, los hilos se crean con `pthread_create()` y se esperan con `pthread_join()`; en Java se usan `Thread`, `start()` y `join()`. En ambos lenguajes, el orden de ejecución no está garantizado porque depende del planificador.

Bibliografía

- [1] L. Lamport, *LaTeX: A Document Preparation System*, 2nd ed. Reading, Massachusetts: Addison-Wesley, 1994.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 7th ed. Boston, MA: Prentice Hall, 2011.